DATABASE.md — Arati Business ERP

Document Version: 1.0
Status: Database Architecture Baseline — Draft for Implementation
Primary Database: PostgreSQL
ORM: Prisma (proposed)
Architecture: Modular Monolith
Primary Scope: Distribution + Wholesale + Retail/POS + Inventory ERP
Accounting Integration: TallyPrime
ERP Priority: ERP first; public website later

---

1. Purpose

This document defines the logical and physical database architecture for the ERP.

It is intended to be strong enough to guide:

- PostgreSQL schema design
- Prisma models
- migrations
- repository/data-access implementation
- transaction boundaries
- database constraints
- inventory integrity
- auditability
- integration state
- automated database tests
- ERD generation

This version supersedes the earlier `DATABASE.md` v0.1 baseline.

The schema is deliberately generic. It must not contain special cases for particular brands, suppliers, customers, dealers, products, or models.

> **Important:** where a business policy is not confirmed, this document does not invent one. Such items are marked **TBD — Business Confirmation Required**.

---

2. Database Authority

2.1 PostgreSQL is authoritative

PostgreSQL is the authoritative transactional source of truth for ERP operational state.

It owns:

- organization and branch structure
- users and authorization data
- product and party masters
- pricing
- procurement
- sales
- inventory
- payments
- returns
- fulfilment
- delivery
- installation
- warranty
- audit records
- integration state

The following must never become the authoritative source of ERP inventory or financial state:

- browser state
- frontend cache
- Redis
- queue payloads
- search indexes
- object storage
- TallyPrime
- temporary files

2.2 Tally boundary

TallyPrime is an external accounting system.

ERP transactions must be able to commit without Tally being available.

Tally synchronization is durable, retryable, idempotent, and reconcilable.

---

3. Core Database Principles

1. Use PostgreSQL transactions for multi-record business operations.
2. Use foreign keys for important relationships.
3. Use unique constraints for business identities that must be unique.
4. Use check constraints where practical for simple invariants.
5. Enforce business rules in the application/domain layer and reinforce critical invariants in the database.
6. Never rely on frontend validation for correctness.
7. Never use floating point for money.
8. Preserve posted transaction history.
9. Prefer deactivation over deletion for referenced masters.
10. Do not directly edit inventory balances as a business operation.
11. Treat inventory movements as an append-oriented ledger.
12. Maintain inventory balances as a transactional projection.
13. Protect concurrent stock consumption with database locking/atomicity.
14. Keep integration state separate from core business records.
15. Avoid polymorphic foreign keys where referential integrity is important.
16. Scope tenant/business data by `organization_id`.
17. Use UTC-aware timestamps (`timestamptz`) and store the organization's business timezone separately.
18. Use migrations as the only normal production schema-change mechanism.
19. Keep audit records append-oriented and protected from ordinary modification.
20. Avoid premature database partitioning, sharding, or separate databases.

---

4. PostgreSQL Conventions

4.1 Naming

Use `snake_case`.

Examples:

sales_invoices
sales_invoice_items
product_variants
inventory_movements
payment_allocations

Foreign keys:

organization_id
branch_id
warehouse_id
product_variant_id

Standard timestamps:

created_at
updated_at

Boolean flags:

is_active
is_system_role

4.2 Primary keys

Use UUID primary keys for application entities.

Recommended PostgreSQL type:

uuid

Use database/application-supported UUID generation consistently.

Human-readable document numbers are separate from internal IDs.

Example:

id = UUID
document_number = INV-2026-000123

4.3 Timestamps

Use:

timestamptz

for event and audit timestamps.

Store organization timezone separately.

Do not store business timestamps as unqualified local time.

4.4 Money

Use:

numeric(19,4)

or an approved precision sufficient for the confirmed tax/accounting requirements.

Do not use:

float
real
double precision

for monetary values.

The final currency scale is **TBD — Business Confirmation Required** if local accounting requirements require a different scale.

4.5 Quantities

Use:

numeric(19,6)

as the initial general-purpose quantity precision.

The effective precision for a unit is controlled by the unit master.

Do not assume every stock quantity is an integer.

4.6 Dates

Use:

- `date` for business dates such as invoice date, purchase date, expiry date.
- `timestamptz` for events such as posting, receiving, payment creation, audit events, and synchronization attempts.

---

5. Organization and Location Model

organization
└── branches
└── warehouses
└── warehouse_locations

The initial deployment may use one organization, one branch, and one warehouse.

The schema must not need redesign when additional branches or warehouses are introduced.

5.1 organizations

Column	Type	Notes

id	uuid PK	Internal identifier
code	varchar	Stable organization code
name	varchar	Display name
legal_name	varchar	Legal entity name
gstin	varchar NULL	Tax identifier; exact structure TBD
phone	varchar NULL	
email	varchar NULL	
address_json	jsonb NULL	Structured address
currency_code	varchar(3)	ISO-style currency code
timezone	varchar	IANA timezone
is_active	boolean	Default true
created_at	timestamptz	
updated_at	timestamptz	

Constraints:

UNIQUE(code)
UNIQUE(gstin) where gstin is not null, if confirmed applicable

5.2 branches

Column	Type
id	uuid PK
organization_id	uuid FK
code	varchar
name	varchar
address_json	jsonb NULL
phone	varchar NULL
is_active	boolean
created_at	timestamptz

updated_at	timestamptz

Constraint:

UNIQUE(organization_id, code)

5.3 warehouses

Column	Type
id	uuid PK
branch_id	uuid FK
code	varchar
name	varchar
address_json	jsonb NULL
is_active	boolean
created_at	timestamptz
updated_at	timestamptz

Constraint:

UNIQUE(branch_id, code)

5.4 warehouse_locations

Optional physical subdivisions inside warehouses.

Column	Type
id	uuid PK
warehouse_id	uuid FK
code	varchar
name	varchar
is_active	boolean
created_at	timestamptz

updated_at	timestamptz

Constraint:

UNIQUE(warehouse_id, code)

A warehouse can be used without requiring physical sublocations.

---

6. Users, Roles, Permissions and Access Scope

6.1 users

Column	Type
id	uuid PK
organization_id	uuid FK
name	varchar
email	varchar NULL
phone	varchar NULL
password_hash	text
is_active	boolean
last_login_at	timestamptz NULL
created_at	timestamptz
updated_at	timestamptz

Recommended:

UNIQUE(organization_id, lower(email))

if email login is enabled.

Passwords are never stored in plaintext.

6.2 roles

Column	Type
id	uuid PK
organization_id	uuid FK
name	varchar
description	text NULL
is_system_role	boolean
created_at	timestamptz
updated_at	timestamptz

Constraint:

UNIQUE(organization_id, name)

6.3 permissions

Global permission catalog.

Column	Type
id	uuid PK
code	varchar
name	varchar
description	text NULL

Constraint:

UNIQUE(code)

6.4 role_permissions

Column	Type
role_id	uuid FK
permission_id	uuid FK

Primary key:

(role_id, permission_id)

6.5 user_roles

Column	Type
user_id	uuid FK
role_id	uuid FK

Primary key:

(user_id, role_id)

6.6 user_access_scopes

Object-level organization/branch/warehouse access should not be inferred only from UI state.

Recommended table:

Column	Type	Notes
id	uuid PK	
user_id	uuid FK	
branch_id	uuid FK NULL	
warehouse_id	uuid FK NULL	
scope_type	varchar	ORGANIZATION / BRANCH / WAREHOUSE
created_at	timestamptz	

Exact authorization model is coordinated with `PROTECTION.md` and application authorization design.

---

7. Product Master

brand
category
unit
tax_rule
↓
product
↓

product_variant
  ├── serial_units
  └── batches

7.1 brands

Column	Type
id	uuid PK
organization_id	uuid FK
code	varchar
name	varchar
is_active	boolean
created_at	timestamptz
updated_at	timestamptz

Constraint:

UNIQUE(organization_id, code)

7.2 categories

Column	Type
id	uuid PK
organization_id	uuid FK
parent_id	uuid FK NULL
code	varchar
name	varchar
is_active	boolean
created_at	timestamptz
updated_at	timestamptz

Constraints:

UNIQUE(organization_id, code)

A category may reference another category in the same organization.

7.3 units

Column	Type
id	uuid PK
organization_id	uuid FK
code	varchar
name	varchar
precision	smallint
is_active	boolean
created_at	timestamptz
updated_at	timestamptz

Constraints:

UNIQUE(organization_id, code)
CHECK(precision >= 0)

7.4 tax_rules

Tax structure is intentionally configurable.

Column	Type
id	uuid PK
organization_id	uuid FK
code	varchar
name	varchar
tax_type	varchar
configuration_json	jsonb
is_active	boolean
created_at	timestamptz
updated_at	timestamptz

The exact GST/tax model is **TBD — Business Confirmation Required**.

Avoid prematurely encoding a tax structure that may later conflict with actual invoicing requirements.

7.5 products

A product is the conceptual product family/master.

Column	Type
id	uuid PK
organization_id	uuid FK
code	varchar
name	varchar
brand_id	uuid FK NULL
category_id	uuid FK NULL
unit_id	uuid FK
tracking_type	varchar
hsn_code	varchar NULL
description	text NULL
reorder_level	numeric(19,6) NULL
is_active	boolean
created_at	timestamptz
updated_at	timestamptz

Tracking types:

QUANTITY
SERIAL
BATCH
HYBRID

Constraint:

UNIQUE(organization_id, code)

7.6 product_variants

Use variants where inventory, SKU, barcode, pricing, tax treatment, or tracking differs.

Column	Type
id	uuid PK
product_id	uuid FK
sku	varchar
model	varchar NULL
barcode	varchar NULL
description	text NULL
attributes_json	jsonb NULL
is_active	boolean
created_at	timestamptz
updated_at	timestamptz

Pricing should **not** be permanently stored here as the authoritative current selling price. Prices belong to price lists.

Historical transaction lines store the actual price used.

Recommended indexes:

UNIQUE(product_id, sku)
UNIQUE(organization_id, sku) via product → organization ownership enforcement
UNIQUE(product_id, barcode) where barcode is not null

Because PostgreSQL cannot enforce a cross-table organization uniqueness constraint directly, the application and schema design must ensure SKU uniqueness at the correct scope. If organization-wide SKU uniqueness is mandatory, either:

1. place `organization_id` directly on `product_variants`, or
2. use an organization-scoped unique identity table.

Recommended: include `organization_id` directly on `product_variants` for simpler integrity and indexing.

Revised variant columns therefore include:

organization_id uuid FK
product_id uuid FK

with:

UNIQUE(organization_id, sku)

---

8. Parties

A single party model supports customer, dealer, supplier, distributor and future roles.

8.1 parties

Column	Type
id	uuid PK
organization_id	uuid FK
code	varchar
name	varchar
display_name	varchar NULL
phone	varchar NULL
email	varchar NULL
gstin	varchar NULL
address_json	jsonb NULL
credit_limit	numeric(19,4) NULL
payment_terms_id	uuid FK NULL
is_active	boolean
created_at	timestamptz
updated_at	timestamptz

Constraint:

UNIQUE(organization_id, code)

Credit behavior is **TBD — Business Confirmation Required**.

8.2 party_roles

Column	Type
id	uuid PK
party_id	uuid FK
role_type	varchar

Candidate role types:

CUSTOMER
DEALER
SUPPLIER
DISTRIBUTOR
OTHER

Constraint:

UNIQUE(party_id, role_type)

One party may have multiple roles.

8.3 payment_terms

Column	Type
id	uuid PK
organization_id	uuid FK
code	varchar
name	varchar
due_days	integer
description	text NULL
is_active	boolean

Constraint:

UNIQUE(organization_id, code)
CHECK(due_days >= 0)

---

9. Pricing

9.1 price_lists

Column	Type
id	uuid PK
organization_id	uuid FK
code	varchar
name	varchar
price_type	varchar
valid_from	date NULL
valid_to	date NULL
is_active	boolean
created_at	timestamptz
updated_at	timestamptz

Examples:

RETAIL
WHOLESALE
DEALER
PROMOTIONAL

Constraint:

UNIQUE(organization_id, code)
CHECK(valid_to is null or valid_from is null or valid_to >= valid_from)

9.2 price_list_items

Column	Type
id	uuid PK
price_list_id	uuid FK
product_variant_id	uuid FK
min_quantity	numeric(19,6)
max_quantity	numeric(19,6) NULL
price	numeric(19,4)

created_at	timestamptz
updated_at	timestamptz

Constraints:

CHECK(min_quantity > 0)
CHECK(max_quantity is null or max_quantity >= min_quantity)
CHECK(price >= 0)

Overlapping quantity ranges must be prevented by application validation and, where practical, PostgreSQL exclusion constraints.

9.3 party_price_lists

Column	Type
id	uuid PK
party_id	uuid FK
price_list_id	uuid FK
priority	integer
valid_from	date NULL
valid_to	date NULL

Pricing precedence is **TBD — Business Confirmation Required**.

---

10. Procurement

purchase_order
↓
goods_receipt
↓
purchase_invoice

These are separate business documents.

A purchase order is not assumed to be mandatory.

A goods receipt represents physical stock receipt.

A purchase invoice represents the supplier financial document.

10.1 purchase_orders

Column	Type
id	uuid PK
organization_id	uuid FK
branch_id	uuid FK
supplier_party_id	uuid FK
warehouse_id	uuid FK
document_number	varchar
document_date	date
status	varchar
subtotal	numeric(19,4)
discount_total	numeric(19,4)
tax_total	numeric(19,4)
grand_total	numeric(19,4)
notes	text NULL
created_by	uuid FK
created_at	timestamptz
updated_at	timestamptz

Constraint:

UNIQUE(organization_id, document_number)

10.2 purchase_order_items

Column	Type
id	uuid PK
purchase_order_id	uuid FK
product_variant_id	uuid FK
quantity	numeric(19,6)

unit_price	numeric(19,4)
discount_amount	numeric(19,4)
tax_amount	numeric(19,4)
line_total	numeric(19,4)

10.3 goods_receipts

Column	Type
id	uuid PK
organization_id	uuid FK
branch_id	uuid FK
warehouse_id	uuid FK
supplier_party_id	uuid FK
purchase_order_id	uuid FK NULL
document_number	varchar
received_at	timestamptz
status	varchar
notes	text NULL
created_by	uuid FK
created_at	timestamptz
updated_at	timestamptz

Constraint:

UNIQUE(organization_id, document_number)

10.4 goods_receipt_items

Column	Type
id	uuid PK
goods_receipt_id	uuid FK

purchase_order_item_id	uuid FK NULL
product_variant_id	uuid FK
quantity	numeric(19,6)
unit_cost	numeric(19,4)

Do **not** store only one `batch_id` or one `serial_unit_id` directly on this line.

A receipt line may contain:

- multiple batches
- multiple serials
- mixed allocations

Therefore explicit allocation tables are required.

10.5 goods_receipt_batch_allocations

Column	Type
id	uuid PK
goods_receipt_item_id	uuid FK
batch_id	uuid FK
quantity	numeric(19,6)

Constraint:

UNIQUE(goods_receipt_item_id, batch_id)

The sum of allocation quantities must equal the receipt-line quantity for batch-tracked stock.

10.6 goods_receipt_serial_allocations

Column	Type
id	uuid PK
goods_receipt_item_id	uuid FK
serial_unit_id	uuid FK

Constraint:

UNIQUE(goods_receipt_item_id, serial_unit_id)

The count of serial allocations must equal the receipt-line quantity for serial-tracked stock.

10.7 purchase_invoices

A purchase invoice may cover one or more receipts, or may exist without a receipt depending on workflow.

Therefore do not model the relationship as a mandatory single `goods_receipt_id`.

Column	Type
id	uuid PK
organization_id	uuid FK
branch_id	uuid FK
supplier_party_id	uuid FK
purchase_order_id	uuid FK NULL
supplier_invoice_number	varchar NULL
document_number	varchar
invoice_date	date
status	varchar
subtotal	numeric(19,4)
discount_total	numeric(19,4)
tax_total	numeric(19,4)
grand_total	numeric(19,4)
created_by	uuid FK
created_at	timestamptz
updated_at	timestamptz

Constraint:

UNIQUE(organization_id, document_number)

If supplier invoice numbers must be unique per supplier, use:

UNIQUE(organization_id, supplier_party_id, supplier_invoice_number)

only if confirmed.

10.8 purchase_invoice_items

Column	Type
id	uuid PK
purchase_invoice_id	uuid FK
product_variant_id	uuid FK
quantity	numeric(19,6)
unit_price	numeric(19,4)
discount_amount	numeric(19,4)
tax_amount	numeric(19,4)
line_total	numeric(19,4)

10.9 purchase_invoice_receipt_allocations

Needed when one purchase invoice is related to multiple receipts or one receipt is invoiced later.

Column	Type
id	uuid PK
purchase_invoice_id	uuid FK
goods_receipt_id	uuid FK
allocated_amount	numeric(19,4) NULL

The exact accounting/quantity matching policy is **TBD — Business Confirmation Required**.

---

11. Sales

quotation
↓
sales_order
↓

sales_invoice

Conversion relationships must preserve traceability.

POS uses the same sales engine and database model.

11.1 quotations

Column	Type
id	uuid PK
organization_id	uuid FK
branch_id	uuid FK
party_id	uuid FK
document_number	varchar
quotation_date	date
valid_until	date NULL
status	varchar
subtotal	numeric(19,4)
discount_total	numeric(19,4)
tax_total	numeric(19,4)
grand_total	numeric(19,4)
notes	text NULL
created_by	uuid FK
created_at	timestamptz
updated_at	timestamptz

Constraint:

UNIQUE(organization_id, document_number)

Quotation does not reduce physical stock by default.

11.2 quotation_items

Column	Type
id	uuid PK
quotation_id	uuid FK
product_variant_id	uuid FK
quantity	numeric(19,6)
unit_price	numeric(19,4)
discount_amount	numeric(19,4)
tax_amount	numeric(19,4)
line_total	numeric(19,4)

11.3 sales_orders

Column	Type
id	uuid PK
organization_id	uuid FK
branch_id	uuid FK
party_id	uuid FK
warehouse_id	uuid FK
quotation_id	uuid FK NULL
document_number	varchar
order_date	date
status	varchar
subtotal	numeric(19,4)
discount_total	numeric(19,4)
tax_total	numeric(19,4)
grand_total	numeric(19,4)
notes	text NULL
created_by	uuid FK

created_at	timestamptz
updated_at	timestamptz

Constraint:

UNIQUE(organization_id, document_number)

Stock reservation is **TBD — Business Confirmation Required**.

11.4 sales_order_items

Column	Type
id	uuid PK
sales_order_id	uuid FK
quotation_item_id	uuid FK NULL
product_variant_id	uuid FK
quantity	numeric(19,6)
unit_price	numeric(19,4)
discount_amount	numeric(19,4)
tax_amount	numeric(19,4)
line_total	numeric(19,4)

Line-level conversion links are preferred over assuming the whole document converts as one unit.

11.5 sales_invoices

Column	Type
id	uuid PK
organization_id	uuid FK
branch_id	uuid FK
warehouse_id	uuid FK
party_id	uuid FK
sale_type	varchar

sales_order_id	uuid FK NULL
document_number	varchar
invoice_date	date
status	varchar
subtotal	numeric(19,4)
discount_total	numeric(19,4)
tax_total	numeric(19,4)
grand_total	numeric(19,4)
created_by	uuid FK
created_at	timestamptz
updated_at	timestamptz

Do not store `amount_paid` and `amount_due` as independently editable source-of-truth columns.

They should be calculated from authoritative invoice totals and payment allocations, or maintained as a carefully controlled projection.

Sale types:

RETAIL
WHOLESALE
DEALER
OTHER

Constraint:

UNIQUE(organization_id, document_number)

11.6 sales_invoice_items

Column	Type
id	uuid PK
sales_invoice_id	uuid FK
sales_order_item_id	uuid FK NULL

product_variant_id	uuid FK
quantity	numeric(19,6)
unit_price	numeric(19,4)
discount_amount	numeric(19,4)
tax_amount	numeric(19,4)
line_total	numeric(19,4)

11.7 sales_invoice_batch_allocations

Column	Type
id	uuid PK
sales_invoice_item_id	uuid FK
batch_id	uuid FK
quantity	numeric(19,6)

Constraint:

UNIQUE(sales_invoice_item_id, batch_id)

11.8 sales_invoice_serial_allocations

Column	Type
id	uuid PK
sales_invoice_item_id	uuid FK
serial_unit_id	uuid FK

Constraint:

UNIQUE(sales_invoice_item_id, serial_unit_id)

A serial unit may be allocated to at most one active sale.

Concurrency must prevent double allocation.

---

12. Serial Inventory

Serialized inventory requires lifecycle history, not merely current fields.

12.1 serial_units

Column	Type
id	uuid PK
organization_id	uuid FK
product_variant_id	uuid FK
serial_number	varchar
status	varchar
current_warehouse_id	uuid FK NULL
current_location_id	uuid FK NULL
created_at	timestamptz
updated_at	timestamptz

Constraint:

UNIQUE(organization_id, serial_number)

The uniqueness scope is **TBD — Business Confirmation Required** if supplier/manufacturer serials can legitimately repeat across organizational contexts.

Do not keep direct `purchase_invoice_id` and `sale_invoice_id` fields as the primary lifecycle history.

Purchase, sale, return, transfer, repair and other state changes are represented by authoritative inventory/serial movement history.

12.2 serial_movements

Recommended explicit serial history.

Column	Type
id	uuid PK

organization_id	uuid FK
serial_unit_id	uuid FK
movement_type	varchar
from_warehouse_id	uuid FK NULL
to_warehouse_id	uuid FK NULL
from_location_id	uuid FK NULL
to_location_id	uuid FK NULL
reference_type	varchar
reference_id	uuid
occurred_at	timestamptz
created_by	uuid FK
created_at	timestamptz

This gives a durable lifecycle trail independent of mutable current-state columns.

---

13. Batch Inventory

13.1 batches

Column	Type
id	uuid PK
organization_id	uuid FK
product_variant_id	uuid FK
batch_number	varchar
manufacturing_date	date NULL
expiry_date	date NULL
created_at	timestamptz
updated_at	timestamptz

Constraint:

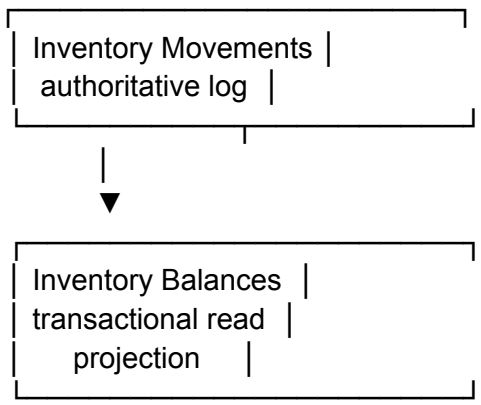
UNIQUE(organization_id, product_variant_id, batch_number)

Expiry handling is **TBD — Business Confirmation Required**.

---

14. Inventory Architecture

Inventory has two layers:



The ledger is authoritative.

The balance table exists for performance and must be updated in the same database transaction as the movement.

14.1 inventory_movements

Recommended schema:

Column	Type
id	uuid PK
organization_id	uuid FK
warehouse_id	uuid FK
location_id	uuid FK NULL
product_variant_id	uuid FK
batch_id	uuid FK NULL
serial_unit_id	uuid FK NULL

movement_type	varchar
quantity	numeric(19,6)
unit_cost	numeric(19,4) NULL
reference_type	varchar
reference_id	uuid
occurred_at	timestamptz
created_by	uuid FK
created_at	timestamptz

Movement types:

OPENING
PURCHASE_IN
SALE_OUT
SALES_RETURN_IN
PURCHASE_RETURN_OUT
TRANSFER_OUT
TRANSFER_IN
ADJUSTMENT_IN
ADJUSTMENT_OUT
DAMAGE_OUT
INTERNAL_USE_OUT
REPAIR_OUT
REPAIR_IN

Final movement catalog may expand after workflow confirmation.

Direction

A movement row should have an unambiguous sign convention.

Recommended: store all quantities as positive magnitudes and use `movement_type` to determine direction.

Do not mix positive/negative conventions arbitrarily.

Reference integrity

`reference_type/reference_id` is a polymorphic relationship and cannot receive a normal PostgreSQL foreign key.

For critical movements, use explicit nullable source foreign keys or dedicated source tables where practical.

At minimum, the application must validate that the referenced source document exists, belongs to the same organization, is in an eligible state, and is authorized to cause the movement.

14.2 inventory_balances

Column	Type
id	uuid PK
organization_id	uuid FK
warehouse_id	uuid FK
location_id	uuid FK NULL
product_variant_id	uuid FK
batch_id	uuid FK NULL
quantity	numeric(19,6)
reserved_quantity	numeric(19,6)
updated_at	timestamptz

`available_quantity` should normally be calculated as:

quantity - reserved_quantity

unless a different confirmed policy is adopted.

Null-safe uniqueness

Because PostgreSQL normally treats `NULL` values as distinct in unique constraints, a simple:

UNIQUE(organization_id, warehouse_id, location_id, product_variant_id, batch_id)

does not safely prevent duplicate rows when `location_id` or `batch_id` is null.

Use a PostgreSQL design such as:

- `UNIQUE NULLS NOT DISTINCT` where supported by the target PostgreSQL version, or
- a carefully designed expression/partial unique index, or
- normalized non-null sentinel/reference rows.

The implementation must guarantee exactly one balance row per logical stock bucket.

14.3 Inventory invariants

The system must guarantee:

1. No stock-affecting operation bypasses the inventory engine.
2. A sale cannot consume unavailable stock.
3. A serialized unit cannot be consumed twice.
4. Batch identity is preserved where batch tracking applies.
5. Transfers do not create or destroy net stock.
6. Adjustments do not overwrite historical movements.
7. Balance projection remains transactionally consistent with the movement ledger.
8. Organization and warehouse scope cannot be crossed accidentally.
9. Historical movements remain immutable.
10. Corrections use compensating movements rather than editing history.

---

15. Stock Transfers

15.1 stock_transfers

Column	Type
id	uuid PK
organization_id	uuid FK
from_warehouse_id	uuid FK
to_warehouse_id	uuid FK
document_number	varchar
transfer_date	date
status	varchar
notes	text NULL
created_by	uuid FK
created_at	timestamptz
updated_at	timestamptz

Constraint:

UNIQUE(organization_id, document_number)

CHECK(from_warehouse_id <> to_warehouse_id)

Whether transfers are immediate or have an in-transit state is **TBD — Business Confirmation Required**.

15.2 stock_transfer_items

Column	Type
id	uuid PK
stock_transfer_id	uuid FK
product_variant_id	uuid FK
quantity	numeric(19,6)

15.3 stock_transfer_batch_allocations

Column	Type
id	uuid PK
stock_transfer_item_id	uuid FK
batch_id	uuid FK
quantity	numeric(19,6)

15.4 stock_transfer_serial_allocations

Column	Type
id	uuid PK
stock_transfer_item_id	uuid FK
serial_unit_id	uuid FK

Serialized and batch-tracked transfers must preserve identity.

---

16. Stock Adjustments

16.1 stock_adjustments

Column	Type
id	uuid PK
organization_id	uuid FK
branch_id	uuid FK
warehouse_id	uuid FK
document_number	varchar
adjustment_date	date
reason	text
status	varchar
created_by	uuid FK
approved_by	uuid FK NULL
created_at	timestamptz
updated_at	timestamptz

Constraint:

UNIQUE(organization_id, document_number)

16.2 stock_adjustment_items

Column	Type
id	uuid PK
stock_adjustment_id	uuid FK
product_variant_id	uuid FK
system_quantity	numeric(19,6)
physical_quantity	numeric(19,6)
difference	numeric(19,6)
reason	text NULL

`difference` may be stored as a derived/auditable value but must never be used to overwrite historical stock.

---

17. Payments and Allocations

Payments are independent records.

One payment may settle multiple documents.

One invoice may be settled by multiple payments.

17.1 payments

Column	Type
id	uuid PK
organization_id	uuid FK
branch_id	uuid FK
party_id	uuid FK
payment_type	varchar
payment_method	varchar
amount	numeric(19,4)
payment_date	date
reference_number	varchar NULL
notes	text NULL
status	varchar
created_by	uuid FK
created_at	timestamptz
updated_at	timestamptz

Candidate payment types:

RECEIPT
PAYMENT
ADVANCE
REFUND

Overpayment behavior is **TBD — Business Confirmation Required**.

17.2 payment_allocations

Do not use a polymorphic `document_type/document_id` table as the long-term integrity model for critical financial allocations.

Recommended explicit allocation tables:

receipt_invoice_allocations

For customer receipts allocated to sales invoices.

Column	Type
id	uuid PK
payment_id	uuid FK
sales_invoice_id	uuid FK
allocated_amount	numeric(19,4)
created_at	timestamptz

supplier_payment_allocations

For supplier payments allocated to purchase invoices.

Column	Type
id	uuid PK
payment_id	uuid FK
purchase_invoice_id	uuid FK
allocated_amount	numeric(19,4)
created_at	timestamptz

If future document types require allocation, add explicit allocation tables instead of making all financial references polymorphic.

Constraints:

CHECK(allocated_amount > 0)

The sum of allocations cannot exceed the eligible payment amount or document balance, subject to confirmed overpayment/advance rules.

---

18. Sales Returns

18.1 sales_returns

Column	Type
id	uuid PK
organization_id	uuid FK
branch_id	uuid FK
warehouse_id	uuid FK
party_id	uuid FK
sales_invoice_id	uuid FK NULL
document_number	varchar
return_date	date
status	varchar
reason	text
subtotal	numeric(19,4)
tax_total	numeric(19,4)
grand_total	numeric(19,4)
created_by	uuid FK
created_at	timestamptz
updated_at	timestamptz

Constraint:

UNIQUE(organization_id, document_number)

18.2 sales_return_items

Column	Type
id	uuid PK
sales_return_id	uuid FK
sales_invoice_item_id	uuid FK NULL
product_variant_id	uuid FK
quantity	numeric(19,6)
unit_price	numeric(19,4)
tax_amount	numeric(19,4)
disposition	varchar

Candidate dispositions:

RESELLABLE
REPAIR_REQUIRED
DAMAGED
SCRAP
SUPPLIER_RETURN

18.3 sales_return_batch_allocations

Column	Type
id	uuid PK
sales_return_item_id	uuid FK
batch_id	uuid FK
quantity	numeric(19,6)

18.4 sales_return_serial_allocations

Column	Type
id	uuid PK
sales_return_item_id	uuid FK
serial_unit_id	uuid FK

Returns must preserve the identity of serialized/batch stock.

Return eligibility and inspection policy are **TBD — Business Confirmation Required**.

---

19. Purchase Returns

19.1 purchase_returns

Column	Type
id	uuid PK
organization_id	uuid FK
branch_id	uuid FK
warehouse_id	uuid FK
supplier_party_id	uuid FK
purchase_invoice_id	uuid FK NULL
document_number	varchar
return_date	date
status	varchar
reason	text
subtotal	numeric(19,4)
tax_total	numeric(19,4)
grand_total	numeric(19,4)
created_by	uuid FK
created_at	timestamptz
updated_at	timestamptz

19.2 purchase_return_items

Column	Type

id	uuid PK
purchase_return_id	uuid FK
purchase_invoice_item_id	uuid FK NULL
product_variant_id	uuid FK
quantity	numeric(19,6)
unit_price	numeric(19,4)
tax_amount	numeric(19,4)

Batch and serial allocation tables should mirror sales returns where tracking requires them.

---

20. Delivery

Delivery is a fulfilment process, not a replacement for sales.

20.1 deliveries

Column	Type
id	uuid PK
organization_id	uuid FK
branch_id	uuid FK
sales_order_id	uuid FK NULL
sales_invoice_id	uuid FK NULL
party_id	uuid FK
assigned_user_id	uuid FK NULL
status	varchar
scheduled_at	timestamptz NULL
delivered_at	timestamptz NULL
delivery_address_json	jsonb
notes	text NULL

created_at	timestamptz
updated_at	timestamptz

Candidate statuses:

PENDING
READY
OUT_FOR_DELIVERY
DELIVERED
CANCELLED

20.2 delivery_items

A delivery must support multiple items and partial fulfilment.

Column	Type
id	uuid PK
delivery_id	uuid FK
sales_order_item_id	uuid FK NULL
sales_invoice_item_id	uuid FK NULL
product_variant_id	uuid FK
quantity	numeric(19,6)

For serialized products, add:

20.3 delivery_serial_allocations

Column	Type
id	uuid PK
delivery_item_id	uuid FK
serial_unit_id	uuid FK

Delivery does not automatically mean stock is sold. The exact stock/dispatch timing is a workflow decision and must remain consistent with the confirmed sales process.

---

21. Installation

Installation is optional for MVP but the schema should support it without redesign.

21.1 installations

Column	Type
id	uuid PK
organization_id	uuid FK
delivery_id	uuid FK NULL
sales_invoice_id	uuid FK NULL
party_id	uuid FK
assigned_user_id	uuid FK NULL
status	varchar
scheduled_at	timestamptz NULL
completed_at	timestamptz NULL
notes	text NULL
created_at	timestamptz
updated_at	timestamptz

21.2 installation_items

Column	Type
id	uuid PK
installation_id	uuid FK
product_variant_id	uuid FK
quantity	numeric(19,6)
serial_unit_id	uuid FK NULL

Installation may later need technician, checklist, location, service charge and proof-of-completion fields.

Do not add these until required by the confirmed workflow.

---

22. Warranty

22.1 warranties

Column	Type
id	uuid PK
organization_id	uuid FK
party_id	uuid FK
product_variant_id	uuid FK
serial_unit_id	uuid FK NULL
sales_invoice_id	uuid FK NULL
start_date	date
expiry_date	date
warranty_type	varchar
terms	text NULL
status	varchar
created_at	timestamptz
updated_at	timestamptz

For serialized products, `serial_unit_id` should normally be populated.

Warranty start-date policy is **TBD — Business Confirmation Required**.

---

23. Document Numbering

Document numbers must be concurrency-safe.

Recommended supporting table:

document_sequences

Column	Type
id	uuid PK
organization_id	uuid FK
branch_id	uuid FK NULL
document_type	varchar
prefix	varchar NULL
current_number	bigint
created_at	timestamptz
updated_at	timestamptz

Constraint:

UNIQUE(organization_id, branch_id, document_type)

Sequence scope, prefixes, fiscal-year behavior, reset rules and gap policy are:

TBD — Business Confirmation Required

The implementation must use row locking or an equivalent atomic mechanism so concurrent requests cannot generate duplicate document numbers.

---

24. Transaction Lifecycle

Documents should use explicit controlled states.

A generic candidate lifecycle is:

DRAFT
↓
CONFIRMED / POSTED
↓
COMPLETED

With controlled correction paths:

POSTED

├── CANCELLED
├── REVERSED
└── CORRECTED THROUGH NEW TRANSACTION

Exact state names vary by document.

Rules:

- Draft records may be editable according to permissions.
- Posted records are historical facts.
- Posted records must not be silently deleted.
- Cancellation/reversal must preserve traceability.
- Inventory-affecting cancellation must create the appropriate compensating movement.
- Financial correction must preserve the original transaction and its audit trail.
- Tally synchronization state must not redefine ERP document truth.

---

25. Audit Log

25.1 audit_logs

Column	Type
id	uuid PK
organization_id	uuid FK
user_id	uuid FK NULL
action	varchar
entity_type	varchar
entity_id	uuid
before_json	jsonb NULL
after_json	jsonb NULL
reason	text NULL
correlation_id	uuid NULL
created_at	timestamptz

Audit records should be append-only from the application's perspective.

High-risk actions must include enough context to answer:

- who
- what
- when
- where
- before
- after
- why
- source request/correlation

Avoid storing secrets, passwords, authentication tokens, or unnecessary sensitive payloads.

---

26. Integration Outbox

The architecture requires external integration to be decoupled from core transactions.

26.1 integration_events

Recommended durable outbox table:

Column	Type
id	uuid PK
organization_id	uuid FK
event_type	varchar
aggregate_type	varchar
aggregate_id	uuid
payload_json	jsonb
status	varchar
available_at	timestamptz
processed_at	timestamptz NULL
attempt_count	integer
last_error	text NULL
created_at	timestamptz

Candidate statuses:

PENDING
PROCESSING
PROCESSED
FAILED

A core transaction creates its integration event in the same PostgreSQL transaction as the business change.

A worker later processes the event.

This prevents the failure mode where:

ERP transaction commits
BUT
integration job is never created

The outbox is therefore part of transactional reliability.

---

27. Tally Integration

27.1 tally_connections

Column	Type
id	uuid PK
organization_id	uuid FK
name	varchar
configuration_reference	text
is_active	boolean
last_success_at	timestamptz NULL
created_at	timestamptz
updated_at	timestamptz

Credentials/secrets are stored through the approved secret-management mechanism, not plaintext database columns.

27.2 tally_mappings

Column	Type
id	uuid PK
organization_id	uuid FK
entity_type	varchar
erp_entity_id	uuid
tally_entity_type	varchar
tally_reference	varchar
status	varchar
created_at	timestamptz
updated_at	timestamptz

Recommended unique identity:

UNIQUE(organization_id, entity_type, erp_entity_id, tally_entity_type)

27.3 tally_sync_jobs

The outbox represents the durable event. The sync job represents the integration attempt/state.

Column	Type
id	uuid PK
organization_id	uuid FK
integration_event_id	uuid FK NULL
entity_type	varchar
entity_id	uuid
operation	varchar
status	varchar
idempotency_key	varchar
attempt_count	integer
next_attempt_at	timestamptz NULL
last_error	text NULL

tally_reference	varchar NULL
created_at	timestamptz
updated_at	timestamptz

Constraint:

UNIQUE(organization_id, idempotency_key)

Candidate statuses:

PENDING
PROCESSING
SYNCED
FAILED
RETRYING

27.4 tally_sync_attempts

Column	Type
id	uuid PK
sync_job_id	uuid FK
attempt_number	integer
started_at	timestamptz
completed_at	timestamptz NULL
status	varchar
request_reference	varchar NULL
response_reference	varchar NULL
error_message	text NULL
created_at	timestamptz

Do not indiscriminately store raw Tally request/response payloads.

Sensitive or high-volume payload retention must follow the integration/security policy.

---

28. Notifications

notifications

Column	Type
id	uuid PK
organization_id	uuid FK
user_id	uuid FK
type	varchar
title	varchar
message	text
read_at	timestamptz NULL
created_at	timestamptz

Notifications are not the authoritative source for business state.

---

29. Attachments

attachments

Column	Type
id	uuid PK
organization_id	uuid FK
entity_type	varchar
entity_id	uuid
file_name	varchar
storage_key	text
mime_type	varchar
file_size	bigint

checksum	varchar NULL
uploaded_by	uuid FK
created_at	timestamptz

Actual binary files should normally live in object storage.

`entity_type/entity_id` is acceptable here because attachment ownership is an application-level association, but it must be validated by the application.

Do not store uploaded files directly in PostgreSQL unless a specific requirement justifies it.

---

30. Reporting and Derived Balances

The database should distinguish between:

Authoritative transactional data

- invoices
- invoice lines
- purchase documents
- payments
- payment allocations
- inventory movements
- returns
- transfers

Derived/projection data

- inventory balances
- receivable balances
- payable balances
- dashboard totals
- search indexes
- cached summaries

Derived data must be rebuildable from authoritative records wherever practical.

For example:

Sales invoice balance
  =
invoice grand total
  -

eligible receipt allocations

Do not make a manually editable `amount_due` field the only source of truth.

Likewise:

inventory balance
  =
authoritative movement history
  +
controlled reservation state

with the balance table acting as the fast operational projection.

---

31. Referential Integrity Strategy

31.1 Explicit foreign keys

Use explicit FKs for:

- organization
- branch
- warehouse
- product
- variant
- party
- invoice
- payment
- return
- transfer
- serial
- batch
- user
- audit
- integration event

31.2 Polymorphic references

Avoid polymorphic `entity_type/entity_id` where a strong FK is practical.

Acceptable selective uses include:

- audit entity reference
- attachment owner

- certain integration/event references

Critical financial and inventory relationships should use explicit FKs or dedicated allocation tables.

31.3 Organization consistency

A foreign key alone does not guarantee that related records belong to the same organization.

For example:

sales_invoice.organization_id
sales_invoice.party_id

does not by itself guarantee that the party belongs to that organization.

The implementation must enforce organization consistency through one or more of:

1. composite foreign keys including `organization_id`;
2. organization-scoped relationship tables;
3. database triggers where justified;
4. application/domain validation for relationships that cannot be expressed cleanly.

Recommended for critical transactional tables: include `organization_id` on child records and use composite uniqueness/FK patterns where they materially improve integrity.

Do not duplicate organization IDs blindly; every duplication must serve an integrity/indexing purpose.

---

32. Delete and Retention Strategy

32.1 Masters

Referenced masters should normally be deactivated:

Product → inactive
Party → inactive
Brand → inactive
Category → inactive
Price list → inactive
Warehouse → inactive

32.2 Transactions

Do not hard-delete posted:

- sales invoices
- purchase invoices
- inventory movements
- payments
- returns
- transfers
- adjustments
- audit records

Use controlled cancellation/reversal/correction.

32.3 Drafts

Draft records may be deleted if:

- they have no posted effect,
- they are not referenced by another historical record,
- the user has permission,
- retention requirements permit deletion.

Exact retention policy is **TBD — Business Confirmation Required**.

---

33. Concurrency and Transaction Patterns

33.1 Sale transaction

A stock-affecting sale should be performed as one database transaction:

```
BEGIN

  authenticate/authorize
  validate document state
  validate party
  validate product/variant
  resolve price
  validate tax
  lock/check relevant inventory rows
```

validate available stock
validate serial/batch allocations

create sales invoice
create invoice items
create serial/batch allocations

create inventory movements
update inventory balances

create payment and allocations if applicable

create integration event

COMMIT

Tally processing occurs after commit.

33.2 Purchase receipt transaction

BEGIN

validate supplier
validate warehouse
validate PO linkage if present
validate quantities
create receipt
create batch/serial allocations
create inventory movements
update inventory balances
create integration event if required

COMMIT

33.3 Transfer transaction

For immediate transfer:

BEGIN

lock source inventory
validate available quantity
validate batch/serial identity

create transfer
create transfer items

create TRANSFER_OUT
create TRANSFER_IN

update source balance
update destination balance

COMMIT

For in-transit transfers, a future explicit state/model may be introduced.

33.4 Payment transaction

BEGIN

create payment
validate target documents
validate allocation limits
create allocations
create integration event

COMMIT

33.5 Locking strategy

Use PostgreSQL mechanisms such as:

- row-level locks (`FOR UPDATE`)
- atomic conditional updates
- unique constraints
- transaction isolation
- exclusion constraints where appropriate

Do not depend on application memory locks.

---

34. Inventory Concurrency Example

If stock is:

quantity = 1

and two users simultaneously attempt to sell one unit:

Transaction A ──┐
├── database-controlled stock check

Transaction B ──┘

Only one transaction may successfully consume the available unit.

For serialized inventory, the serial row itself should be locked before changing eligibility.

The second transaction must fail cleanly rather than creating negative or duplicated stock.

---

35. Database Constraints

At minimum, implement constraints for:

Identity

organization codes
branch codes
warehouse codes
product codes
variant SKUs
party codes
document numbers
serial numbers
batch identities

Amounts

money >= 0 where negative values are not meaningful
quantity > 0 for movement/allocation lines

Dates

valid_to >= valid_from
expiry_date >= manufacturing_date where both exist

Relationships

FKs
composite FKs where needed
unique allocation identities

Lifecycle

Where practical, constrain status values through PostgreSQL enums or check constraints.

The project should choose one consistent strategy rather than mixing ad hoc representations.

36. Indexing Strategy

Indexes should be driven by actual query patterns.

Initial important indexes:

Organization/master search

products(organization_id, code)
products(organization_id, name)
product_variants(organization_id, sku)
product_variants(organization_id, barcode)
parties(organization_id, code)
parties(organization_id, name)

Inventory

```
inventory_movements(
  organization_id,
  product_variant_id,
  occurred_at
)

inventory_movements(
  organization_id,
  warehouse_id,
  occurred_at
)

inventory_balances(
  organization_id,
  warehouse_id,
  product_variant_id
)

inventory_balances(
  organization_id,
  product_variant_id,
  batch_id
)

serial_units(
  organization_id,
  serial_number
```

)

Documents

sales_invoices(
  organization_id,
  document_number
)

sales_invoices(
  organization_id,
  invoice_date
)

purchase_invoices(
  organization_id,
  document_number
)

purchase_invoices(
  organization_id,
  invoice_date
)

sales_orders(
  organization_id,
  document_number
)

purchase_orders(
  organization_id,
  document_number
)

Payments

payments(
  organization_id,
  party_id,
  payment_date
)

receipt_invoice_allocations(
  sales_invoice_id
)

supplier_payment_allocations(

  purchase_invoice_id
)

Integration

integration_events(
  status,
  available_at
)

tally_sync_jobs(
  status,
  next_attempt_at
)

Do not create indexes simply because a column exists.

Validate indexes using:

- actual query plans
- production-like data
- slow query logs
- expected POS workload

---

37. Search and Case Handling

For case-insensitive identifiers such as email or possibly SKU/barcode, define an explicit normalization strategy.

Do not allow visually different strings to create accidental duplicate identities.

Potential tools:

- normalized lowercase columns
- PostgreSQL functional indexes
- `citext` where appropriate

Barcode/SKU exact matching should generally be deterministic.

Name search may later use PostgreSQL full-text or trigram indexing if required.

Do not introduce Elasticsearch/OpenSearch for the initial ERP unless PostgreSQL search proves insufficient.

---

38. JSONB Usage

JSONB is appropriate for:

- flexible product attributes
- structured addresses
- tax configuration that is genuinely variable
- audit snapshots
- integration payload metadata
- future optional metadata

JSONB must not become an excuse to avoid relational modeling.

Core transactional relationships belong in relational columns/tables.

Bad:

invoice.items_json

Good:

sales_invoices
sales_invoice_items

---

39. Prisma Mapping Guidance

Prisma should map directly to the PostgreSQL relational model.

Guidelines:

1. Use explicit relation fields.
2. Use `Decimal` for monetary and quantity fields.
3. Use UUID strings or native UUID mapping consistently.
4. Keep enums aligned with database/application lifecycle definitions.
5. Do not hide critical relationships inside JSON.
6. Use migrations for all schema changes.
7. Add raw SQL migrations where Prisma cannot express an important PostgreSQL feature cleanly.
8. Document any database trigger, partial index, expression index, exclusion constraint, or `NULLS NOT DISTINCT` constraint that is maintained outside normal Prisma model syntax.

The Prisma schema is an implementation artifact derived from this database architecture, not the business specification itself.

---

40. Recommended Core Table Inventory

The first physical schema should include approximately these core groups.

Organization

organizations
branches
warehouses
warehouse_locations

Identity and authorization

users
roles
permissions
role_permissions
user_roles
user_access_scopes

Product

brands
categories
units
tax_rules
products
product_variants

Parties and pricing

parties
party_roles
payment_terms
price_lists
price_list_items
party_price_lists

Procurement

purchase_orders
purchase_order_items
goods_receipts
goods_receipt_items
goods_receipt_batch_allocations
goods_receipt_serial_allocations

purchase_invoices
purchase_invoice_items
purchase_invoice_receipt_allocations

Sales

quotations
quotation_items
sales_orders
sales_order_items
sales_invoices
sales_invoice_items
sales_invoice_batch_allocations
sales_invoice_serial_allocations

Inventory

serial_units
serial_movements
batches
inventory_movements
inventory_balances
stock_transfers
stock_transfer_items
stock_transfer_batch_allocations
stock_transfer_serial_allocations
stock_adjustments
stock_adjustment_items

Payments

payments
receipt_invoice_allocations
supplier_payment_allocations

Returns

sales_returns
sales_return_items
sales_return_batch_allocations
sales_return_serial_allocations
purchase_returns
purchase_return_items
purchase_return_batch_allocations
purchase_return_serial_allocations

Fulfilment

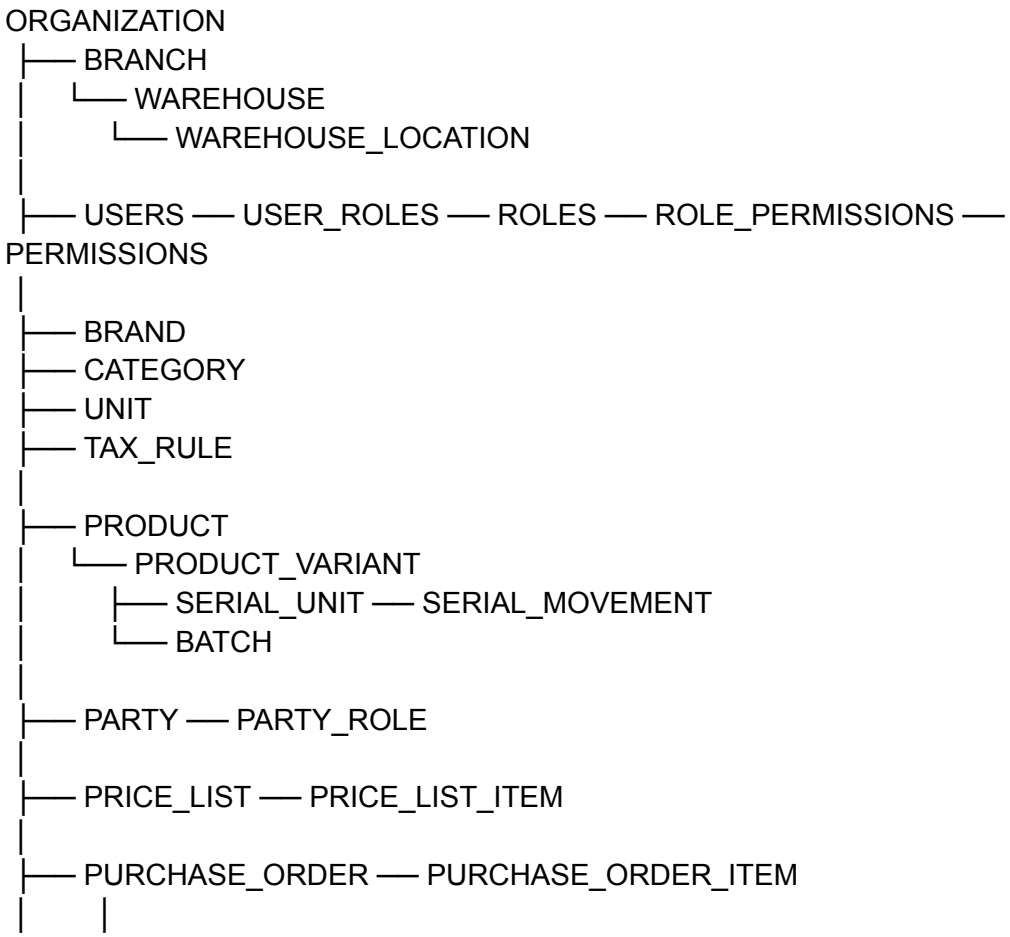
deliveries
delivery_items
delivery_serial_allocations
installations
installation_items
warranties

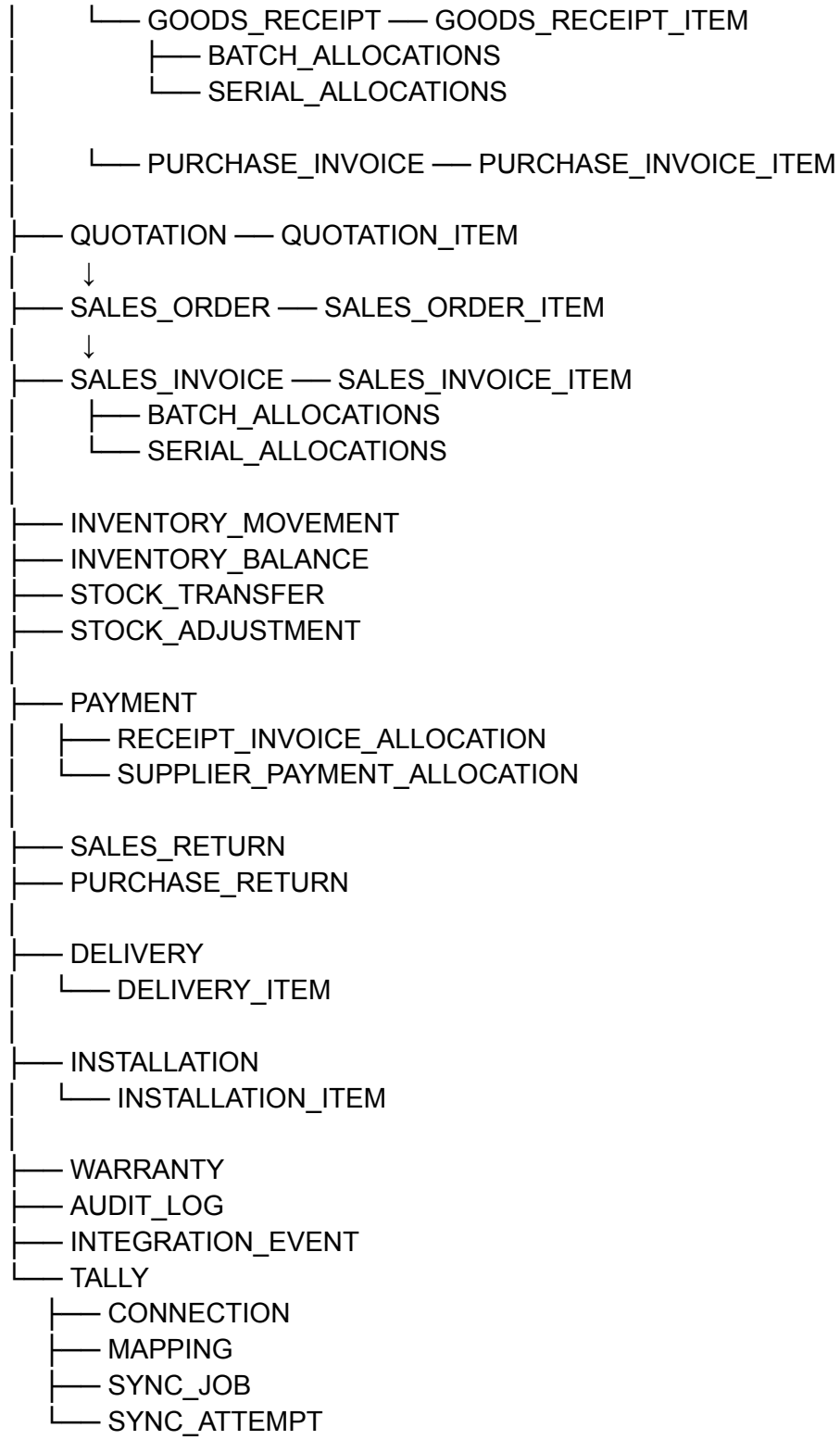
Control and integrations

document_sequences
audit_logs
integration_events
tally_connections
tally_mappings
tally_sync_jobs
tally_sync_attempts
notifications
attachments

---

41. Entity Relationship Overview





---

42. Source-of-Truth Matrix

Data	Authority

Product master	PostgreSQL
Party master	PostgreSQL
Price configuration	PostgreSQL
Purchase documents	PostgreSQL
Sales documents	PostgreSQL
Inventory movements	PostgreSQL
Inventory balance projection	PostgreSQL
Payments	PostgreSQL
Returns	PostgreSQL
Delivery	PostgreSQL
Installation	PostgreSQL
Warranty records	PostgreSQL
Audit	PostgreSQL
Integration state	PostgreSQL
Accounting in Tally	TallyPrime
Uploaded binary files	Object storage
UI state	Browser/frontend only
Cache	Redis only if introduced
Search index	Derived only

---

43. Migration Strategy

Database changes must be version-controlled.

Recommended flow:

Schema change proposal
↓
Review against DATABASE.md
↓
Prisma migration / SQL migration
↓

Migration test
↓
Application integration test
↓
Staging
↓
Production

Never manually modify production tables as the normal deployment process.

For destructive migrations:

1. identify affected data;
2. create backup/rollback plan;
3. verify dependent application code;
4. use expand-and-contract migration where necessary;
5. test against production-like data.

---

44. Seed Data

Development seed data should include generic examples only.

Example:

Organization:
  Demo Distribution Co.

Branches:
  Main Branch

Warehouses:
  Main Warehouse

Products:
  Refrigerator
  Battery
  Air Conditioner

Variants:
  multiple generic model/SKU examples

Parties:
  Customer
  Dealer
  Supplier

Price Lists:
  Retail
  Wholesale
  Dealer

Do not seed production-specific brands or customer identities as architectural assumptions.

---

45. Backup and Recovery Requirements

Database operations must support:

- automated backups
- point-in-time recovery where the deployment environment supports it
- backup retention policy
- restore testing
- migration recovery procedures
- documented recovery responsibilities

A backup that has never been restored is not considered a proven backup.

Recovery objectives:

RPO: TBD
RTO: TBD

These must be confirmed based on the actual deployment/business requirement.

---

46. Database Security

At minimum:

- database must not be publicly exposed unnecessarily;
- application database credentials must be secret-managed;
- least-privilege database roles should be used;
- production credentials must differ from development credentials;
- TLS should be used for database connections where applicable;
- sensitive fields must be protected according to `PROTECTION.md`;
- backups must be protected;
- audit records must be access-controlled;
- application users must not receive unrestricted database access.

The application should use a restricted runtime DB role.

Schema migration privileges should be separated from normal application runtime privileges where operationally practical.

---

47. Observability

Database/application operations should carry a correlation ID.

Important records should be traceable across:

HTTP request
↓
application use case
↓
database transaction
↓
audit event
↓
integration event
↓
Tally sync job
↓
sync attempt

The database should not be filled with high-cardinality tracing payloads unnecessarily.

Use OpenTelemetry-compatible application observability as defined by the architecture/technology documents.

---

48. Data Integrity Tests

Before production, automated tests should verify at least:

Inventory

- cannot sell unavailable quantity;
- cannot sell the same serial twice;
- cannot create duplicate serial identity;
- batch allocations equal line quantities;
- transfer does not create/destroy stock;

- adjustment creates movements rather than rewriting history;
- balance projection matches movement calculations.

Financial

- invoice totals reconcile;
- payment allocations cannot exceed permitted amounts;
- multiple payments can settle one invoice;
- one payment can settle multiple invoices;
- cancellation does not erase history.

Organization

- cross-organization relationships are rejected;
- unauthorized branch/warehouse access is rejected.

Documents

- document numbers cannot collide under concurrency;
- posted documents cannot be silently deleted;
- invalid state transitions fail.

Integration

- business transaction commits if Tally is unavailable;
- integration event exists after successful commit;
- duplicate processing is idempotent;
- failed synchronization is retryable;
- synchronization failure does not corrupt the ERP transaction.

---

49. Database Anti-Patterns

Do not:

- store money as floating point;
- keep inventory only as a manually editable number;
- delete posted invoices;
- allow frontend-only validation;
- use polymorphic relationships for core financial integrity;
- store serial lifecycle only in a few mutable columns;
- put one batch ID directly on a line that may contain multiple batches;
- put one serial ID directly on a line that may contain multiple serials;

- make Tally availability a prerequisite for ERP posting;
- store secrets in plaintext;
- duplicate customer/dealer/supplier master tables without a business reason;
- create separate databases for every module;
- introduce microservice databases before a demonstrated need;
- add Elasticsearch/OpenSearch before PostgreSQL search proves insufficient;
- add Redis/BullMQ merely because it is available;
- create excessive indexes without query evidence;
- use JSON blobs for core transaction relationships;
- make derived balances the only source of truth.

---

50. Business Decisions Still Required

The database can proceed with implementation structure, but the following policies require business confirmation before final production schema freeze:

1. GST/tax structure and tax components.
2. Invoice numbering and fiscal-year reset behavior.
3. Document numbering gap policy.
4. Stock valuation method.
5. Negative stock policy and exception authority.
6. Stock reservation on sales orders.
7. Credit-limit behavior.
8. Payment overpayment/advance behavior.
9. Warranty start-date rule.
10. Return eligibility and inspection workflow.
11. Expiry handling and FEFO/FIFO requirements.
12. Serial lifecycle states.
13. Serial uniqueness scope.
14. Whether variants are mandatory or optional.
15. Purchase order mandatory/optional policy.
16. Multiple receipt/invoice matching policy.
17. Sales order partial invoicing policy.
18. Quotation locking/versioning behavior.
19. Delivery timing relative to invoice/stock deduction.
20. Installation workflow and mandatory/optional status.
21. Transfer in-transit workflow.
22. Damaged/internal-use stock workflow.
23. Approval thresholds.
24. Branch/location access model.
25. Tally master mappings.
26. Tally voucher mappings.
27. Tally synchronization direction.
28. Tally correction/reconciliation rules.

29. Data retention period.
30. Cancellation rules after payment.
31. Cancellation rules after delivery.
32. Cancellation rules after Tally synchronization.
33. Receivable/payable calculation policy.
34. Rounding policy.
35. Currency requirements.
36. Backup RPO.
37. Backup RTO.
38. Required operational reports.

These should become explicit business rules before the physical schema is considered final.

---

51. Definition of Database Done

The database baseline is ready for implementation when:

- every core entity has a defined owner;
- core relationships are modeled relationally;
- important relationships have foreign keys;
- organization/branch/location scope is enforceable;
- money and quantity precision is defined;
- document numbering is concurrency-safe;
- inventory ledger and balance projection are defined;
- serialized inventory preserves identity/history;
- batch allocations support multiple batches per document line;
- financial allocations use explicit integrity-preserving relationships;
- posted transaction immutability is enforced;
- cancellation/reversal is modeled;
- audit logging is defined;
- integration outbox is defined;
- Tally sync state is durable and idempotent;
- indexes match expected query patterns;
- migrations are version-controlled;
- seed data works;
- backup/recovery procedures are documented;
- automated integrity tests pass;
- all critical TBD business policies are resolved.

---

52. Implementation Order

Recommended implementation sequence:

1. Organizations / branches / warehouses
2. Users / roles / permissions / access scope
3. Product master
4. Parties / payment terms
5. Pricing
6. Procurement documents
7. Inventory ledger + balances
8. Serial / batch allocation
9. Sales documents
10. Payments / allocations
11. Returns
12. Transfers / adjustments
13. Delivery
14. Installation
15. Warranty
16. Audit
17. Integration outbox
18. Tally integration
19. Notifications
20. Attachments

The inventory engine should be implemented before sales and purchase posting logic is considered complete.

---

53. Next Database Artifacts

The next database-specific artifacts should be produced in this order:

DATABASE.md v1.0
↓
ERD v1.0
↓
PostgreSQL Physical Schema
↓
PRISMA Schema
↓
Migration Set
↓
Database Seed
↓
Database Integrity Tests
↓
Performance / Query Validation

The ERD should be generated from this document rather than from the old v0.1 schema.

---

54. Cross-Document Alignment

This document must remain aligned with:

- `BRD.md`
- `SRS.md`
- `ARCHITECTURE_v2.0.md`
- `BUSINESS-RULES_v1.0.md`
- `PROTECTION.md`
- `TECHSTACK_v1.1.md`
- `API.md`
- `TALLY-INTEGRATION.md`
- `DESIGN.md`
- `PHASES.md`

If two documents conflict:

1. do not silently choose one;
2. identify the conflict;
3. resolve it through the appropriate business/architecture decision;
4. update all affected documents.

Architecture decisions should be recorded as ADRs when they materially affect system structure.

---

55. Final Database Rules

The following are non-negotiable architectural rules:

1. PostgreSQL is the ERP transactional source of truth.
2. Inventory is controlled through an authoritative movement ledger.
3. Inventory balances are projections, not manually editable truth.
4. Serial identity must be preserved throughout lifecycle movements.
5. Batch identity must be preserved where required.
6. Critical financial allocations must use strong relational integrity.
7. Posted transactions are historical facts.
8. Corrections use controlled compensating transactions.
9. External systems cannot block core ERP commits.

10. Integration events must be durably recorded.
11. Tally synchronization must be retryable and idempotent.
12. Business rules must remain server-enforced.
13. Organization/branch/location access must be enforced.
14. Money must use exact decimal/numeric representation.
15. Database migrations must be version-controlled.
16. The schema must remain generic and must not encode special-case brand/customer logic.
17. Complexity must be justified by a real business or technical requirement.

Document Version: 1.0
Status: Database Architecture Baseline — Draft for Implementation